

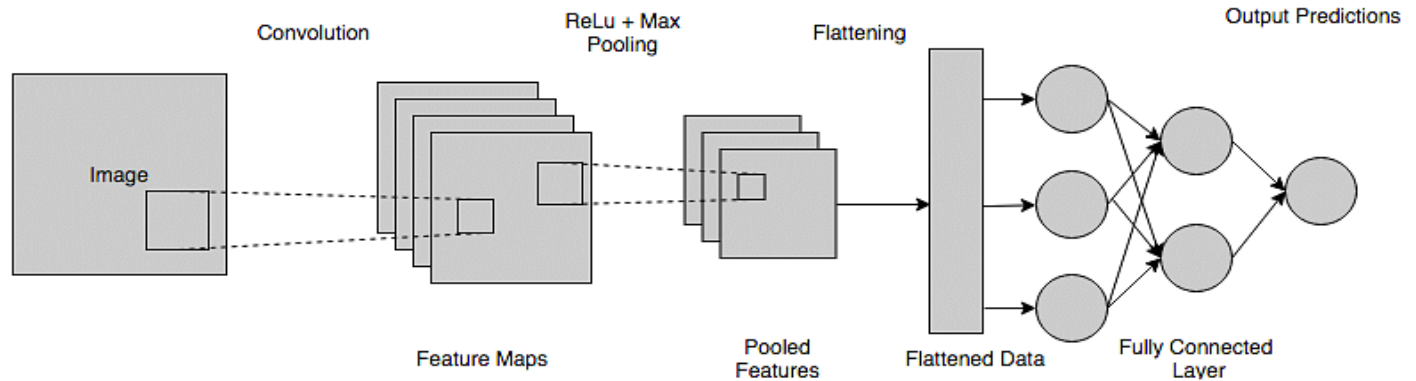


Data Science Lecture 6

Prof. Dr. Rasha Ismail
Dr. Eman Amin

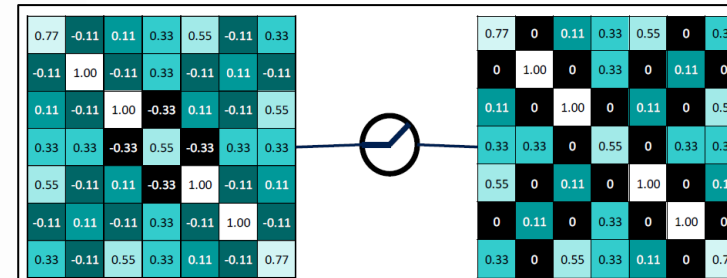
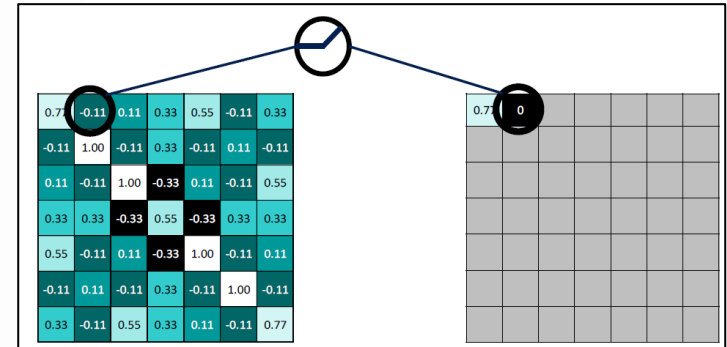
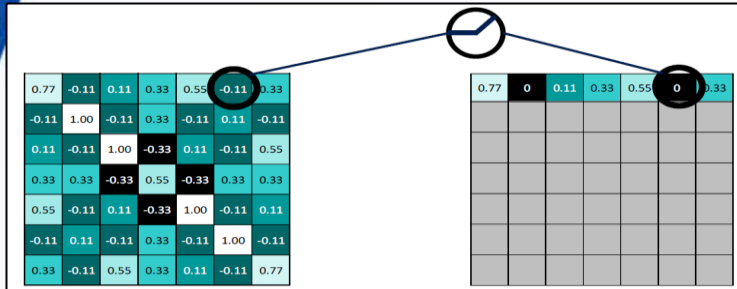
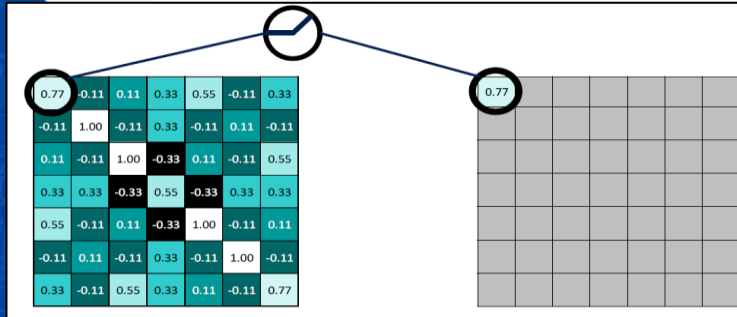
CNN Operations

- There are certain steps/operations that are involved CNN. These can be categorized as follows:
 - Convolution $\checkmark$
 - Activation map (ReLU)
 - Pooling layer
 - Flattening
 - Fully connected layer



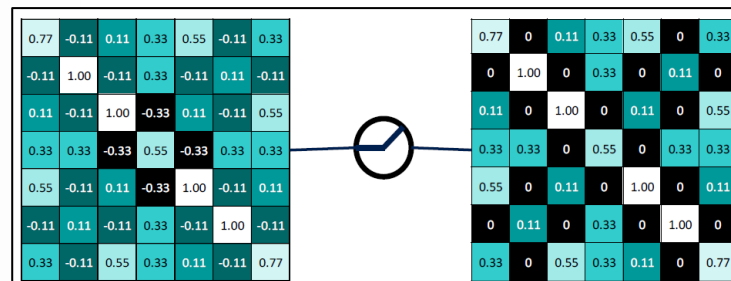
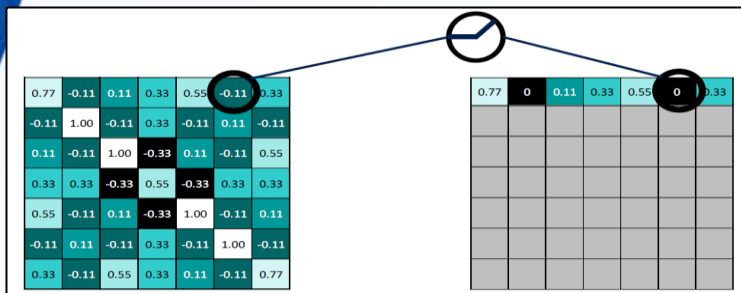
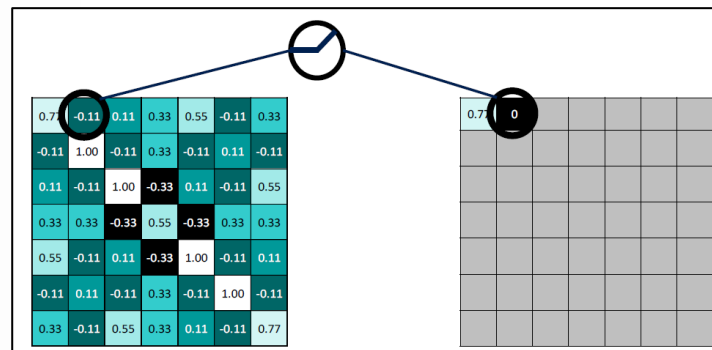
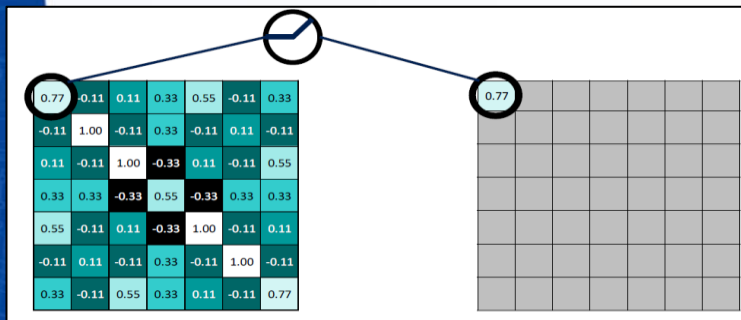
RELU

- ReLU activation function is used to give CNN nonlinear expression ability, so that it can better fit the results, so as to improve the accuracy.



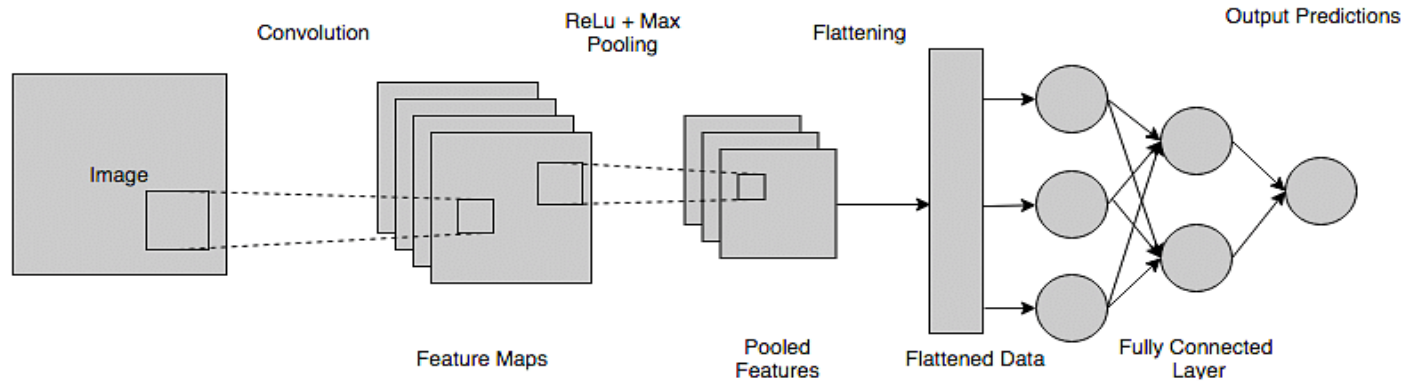
RELU

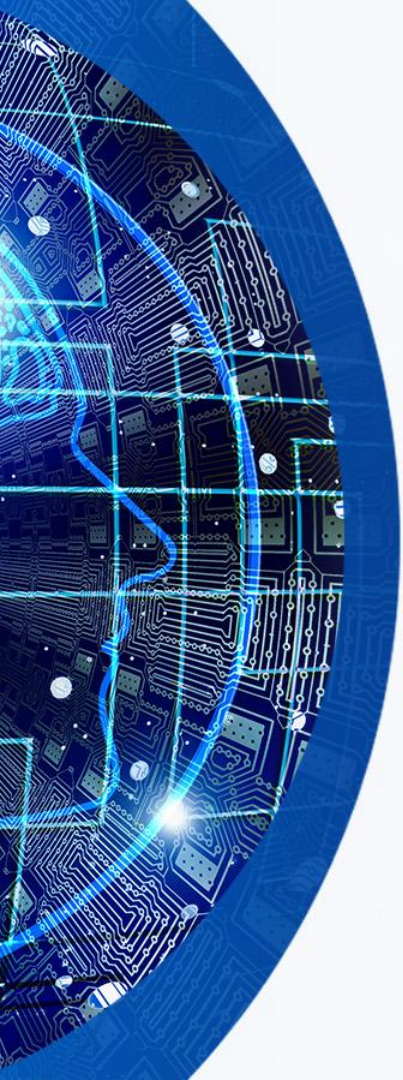
- In this layer, the ReLU activation function is used, and every negative value in the output volume from the convolution layer is replaced with zero. This is done to prevent the values from summing up to zero.



CNN Operations

- There are certain steps/operations that are involved CNN. These can be categorized as follows:
 - Convolution $\checkmark$
 - Activation map (ReLU) $\checkmark$
 - Pooling layer
 - Flattening
 - Fully connected layer



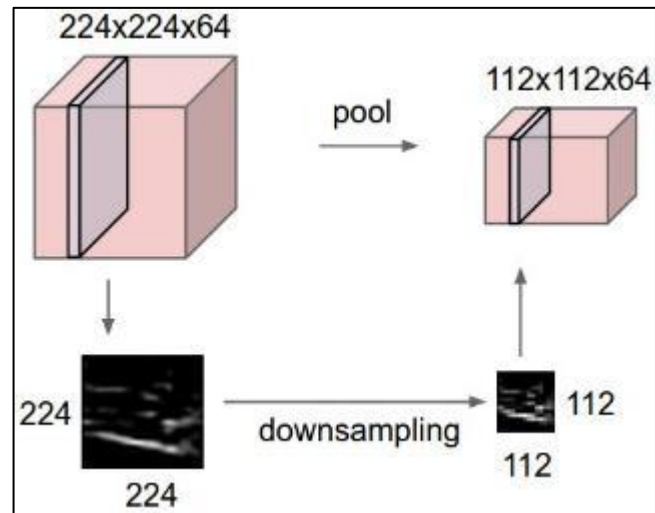
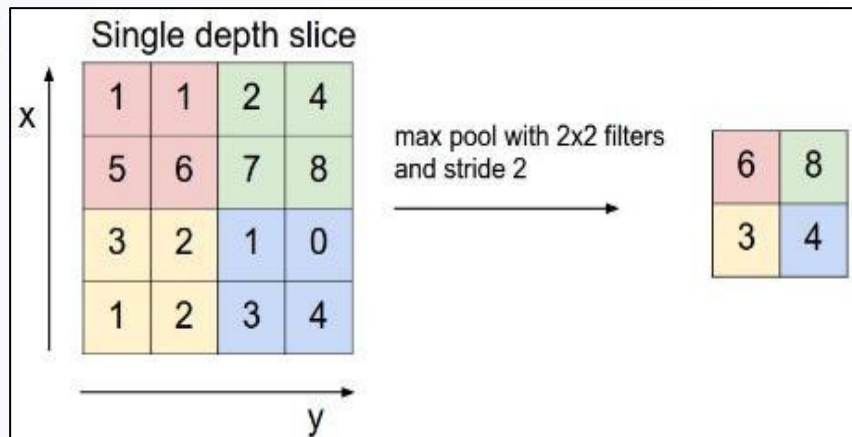


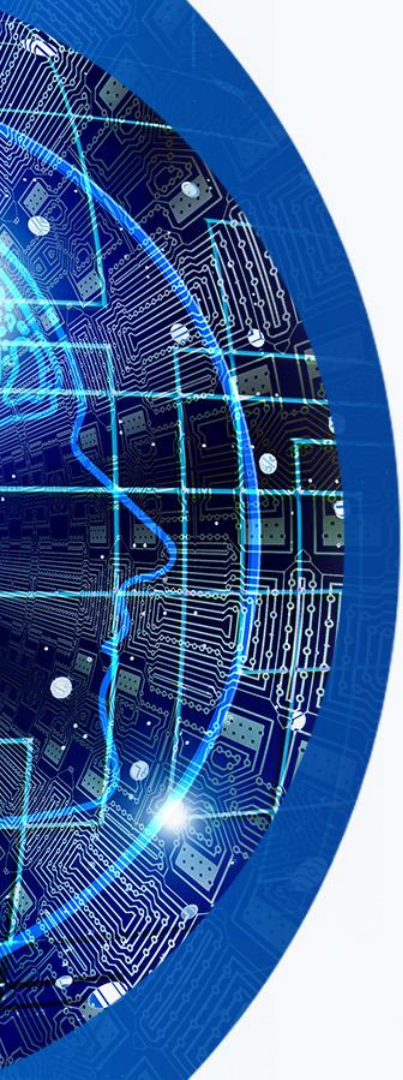
Pooling Layer

- This layer performs **the dimensionality reduction** on the input by reducing the number of parameters, and it also uses a filter to perform this action.
- The only difference is that instead of using a matrix with weights as convolutional layers do, pooling layers perform aggregation on the input pixels.
- Although this operation results in the loss of some data, the **benefits of using pooling** layers include **reducing the complexity of features** making it **less prone to overfitting**, **speeding up the calculations**, and improving the efficiency of CNN.

Pooling Layer

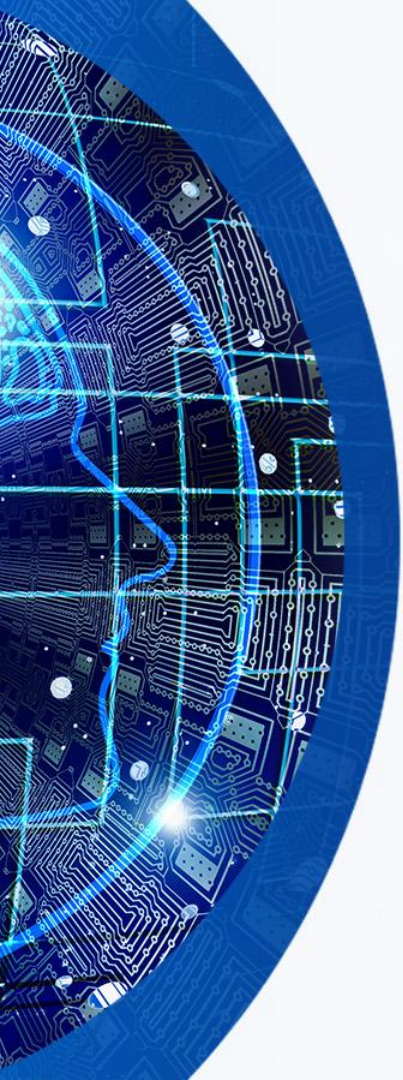
- The pooling layers down-sample the previous layers feature map.





Pooling Layer

- Pooling layers are added in between two convolution layers with the sole purpose of reducing the spatial size of the image representation.
- **The pooling layer has two hyperparameters:**
 - **window size**
 - **stride**
- From each window, we take either the maximum value or the average of the values in the window depending upon the type of pooling being performed.

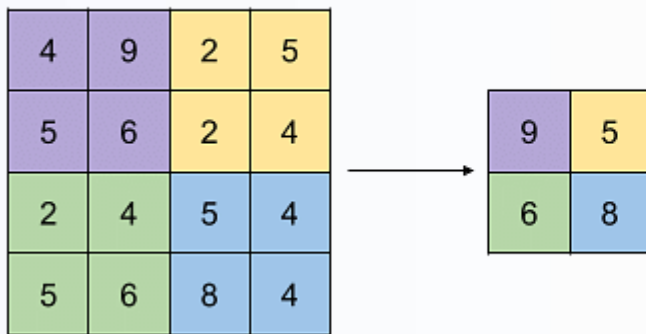


Types of Pooling

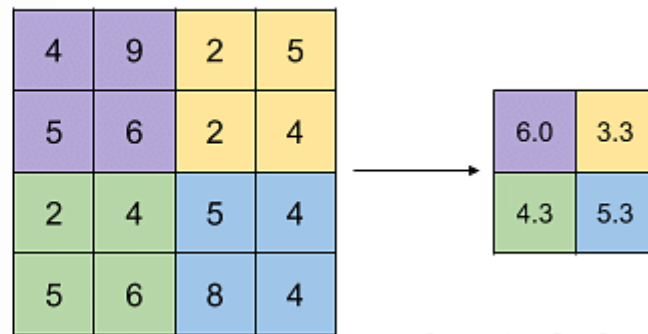
- **Max Pooling** selects the maximum element from each of the windows of the feature map. Thus, after the max-pooling layer, the output would be a feature map containing **the most dominant features** of the previous feature map.
- **Average Pooling** computes the average of the elements present in the region of the feature map covered by the filter. It simply averages the features from the feature map.

Pooling Layer

Max Pooling



Avg Pooling



<https://indoml.com>

Pooling Layer

- This is to decrease the computational power required to process the data through dimensionality reduction.

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

12	20	30	0
8	12	2	0
34	70	37	4
112	100	25	12

max pooling

20	30
112	37

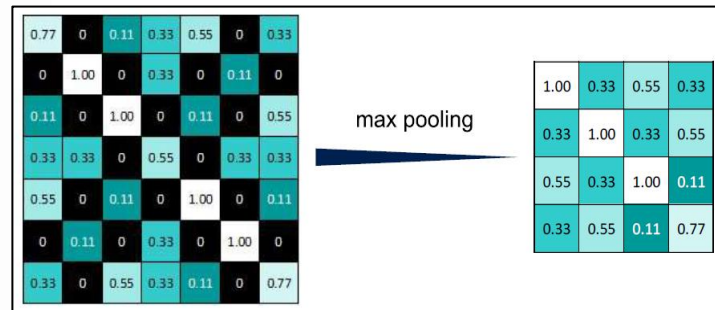
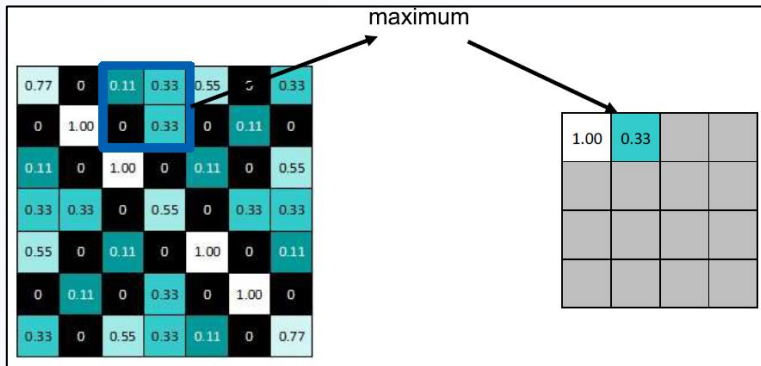
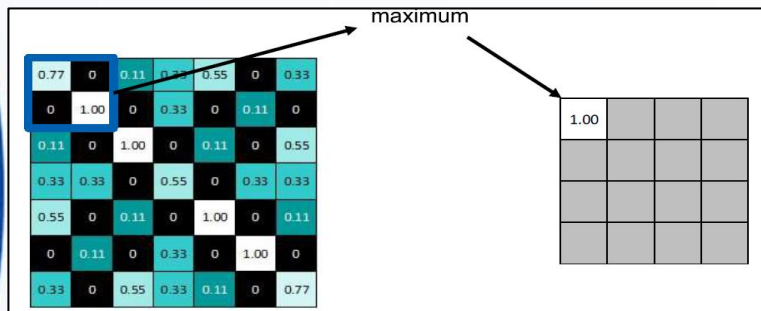
average pooling

13	8
79	20

Example of Max Pooling

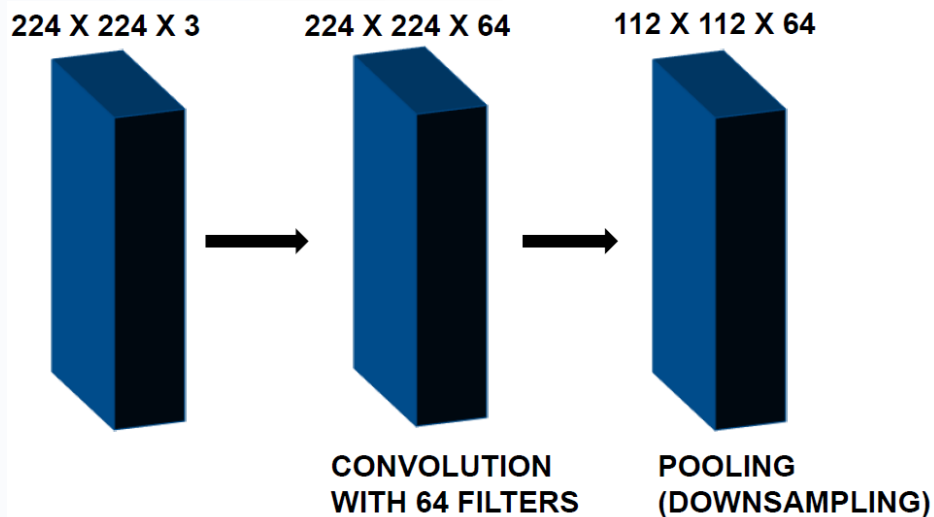
Pooling Example

- Pooling Filter Size = 2×2 , Stride = 2



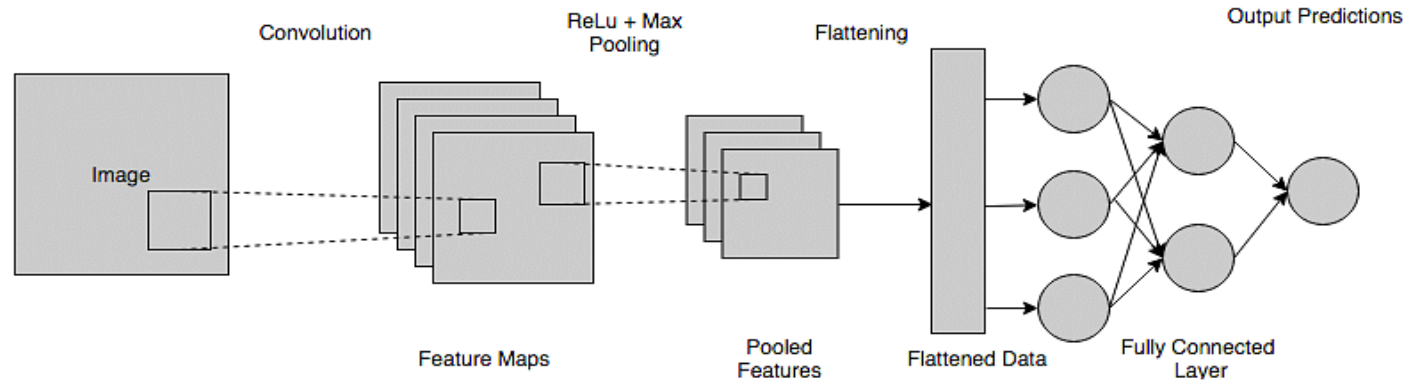
Pooling Layer

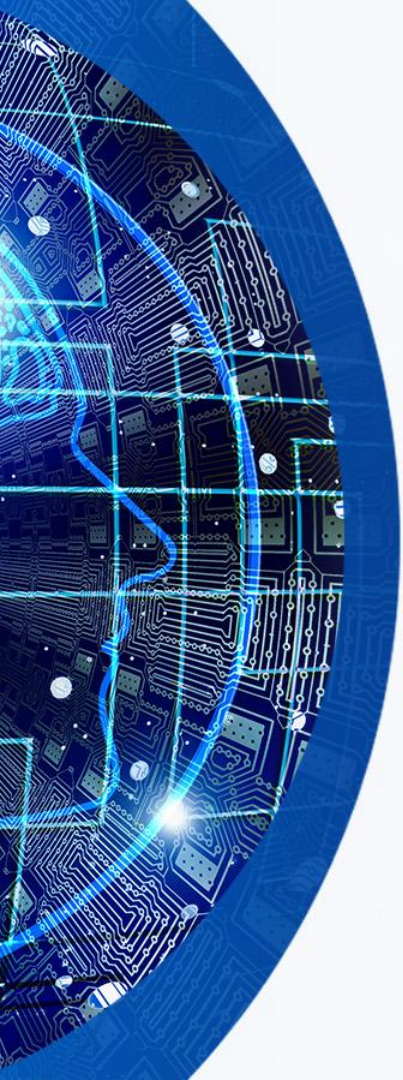
- **Pooling layers don't change the depth of the image. They only reduce its width and height.**



CNN Operations

- There are certain steps/operations that are involved CNN. These can be categorized as follows:
 - Convolution $\checkmark$
 - Activation map (ReLU) $\checkmark$
 - Pooling layer $\checkmark$
 - Flattening
 - Fully connected layer





Flatten Layer

- After going through the previous process, we have successfully enabled the model to understand the features. Moving on, we are going to flatten the final output and feed it to a regular Neural Network for classification purposes
- After multiple convolution layers and downsampling operations, **the 3D representation of the image is converted into a feature vector that is passed into a regular Neural Network to output probabilities.**

Flatten Layer

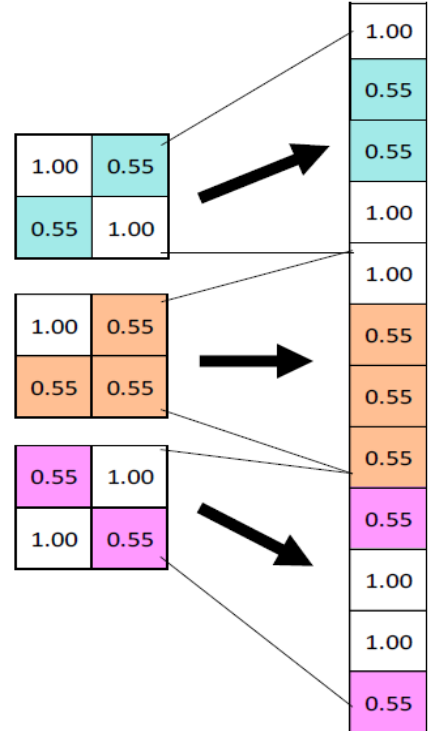
- The rows are concatenated to form a long feature vector. If multiple input layers are present, its rows are also concatenated to form an even longer feature vector.
- simply rearrange the output matrices as a 1-D array.

1	1	0
4	2	1
0	2	1

Pooled Feature Map

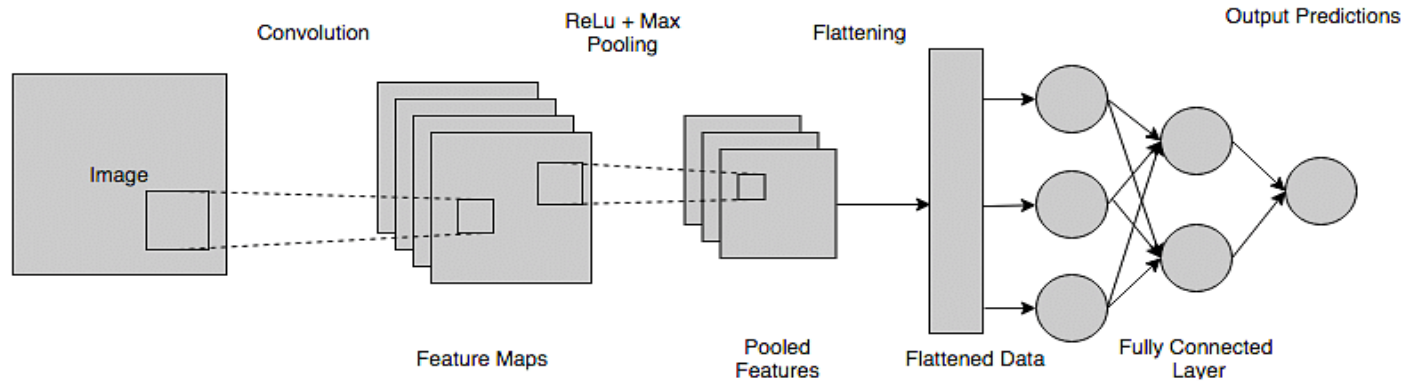
Flattening

1
1
0
4
2
1
0
2
1



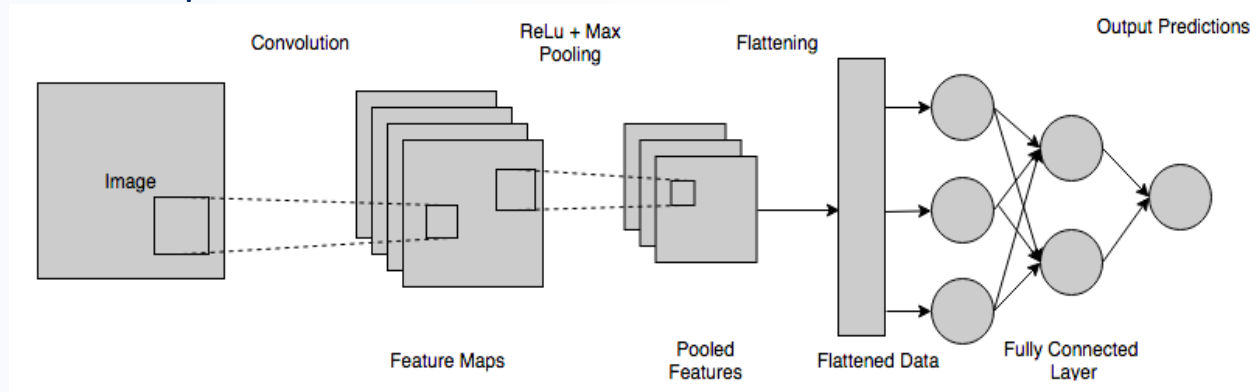
CNN Operations

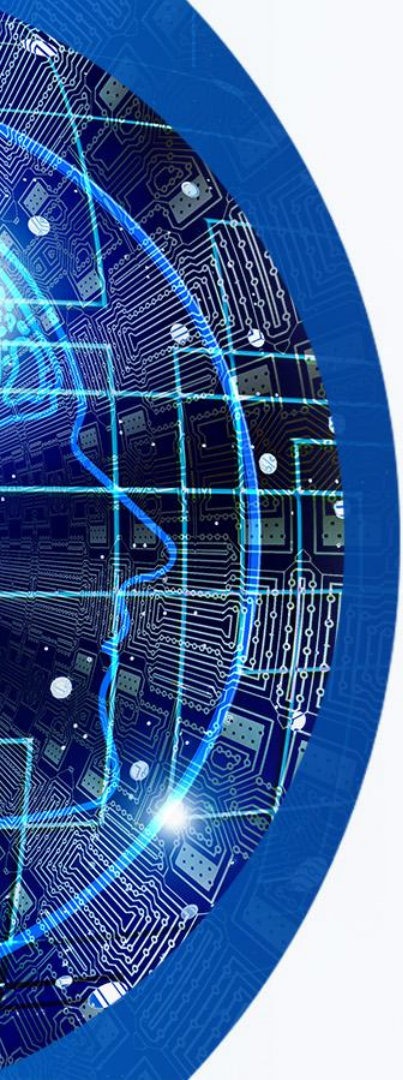
- There are certain steps/operations that are involved CNN. These can be categorized as follows:
 - Convolution $\checkmark$
 - Activation map (RELU) $\checkmark$
 - Pooling layer $\checkmark$
 - Flattening $\checkmark$
 - Fully connected layer



Fully-Connected Layer

- Fully connected layers are the normal flat feed-forward neural network layers.
- The flattened output **is fed to** this fully-connected layer(s) for the final classification task.
- This layer uses a non-linear activation function or softmax activation function to predict the probabilities of classes for final prediction.

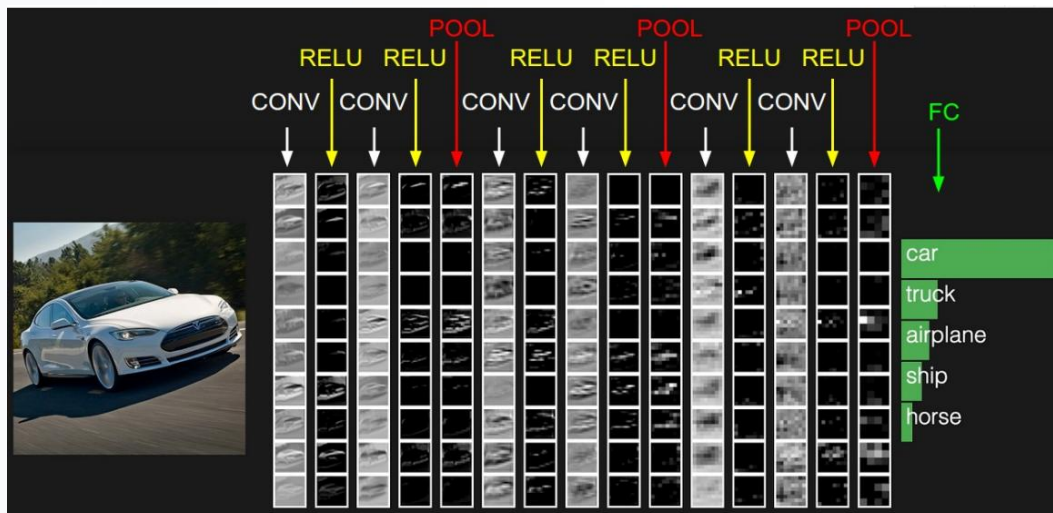
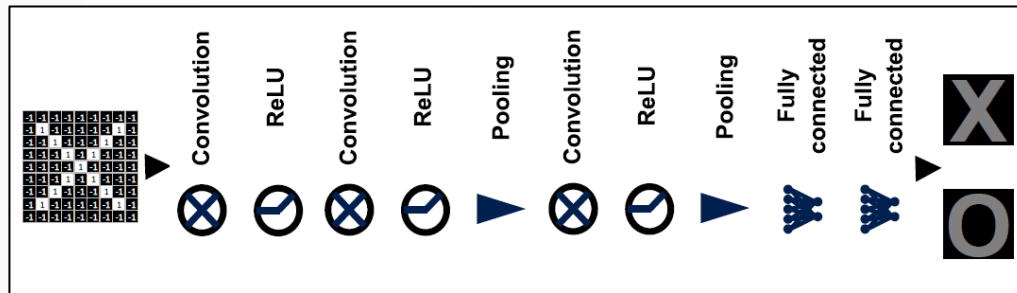




Fully-Connected Layer

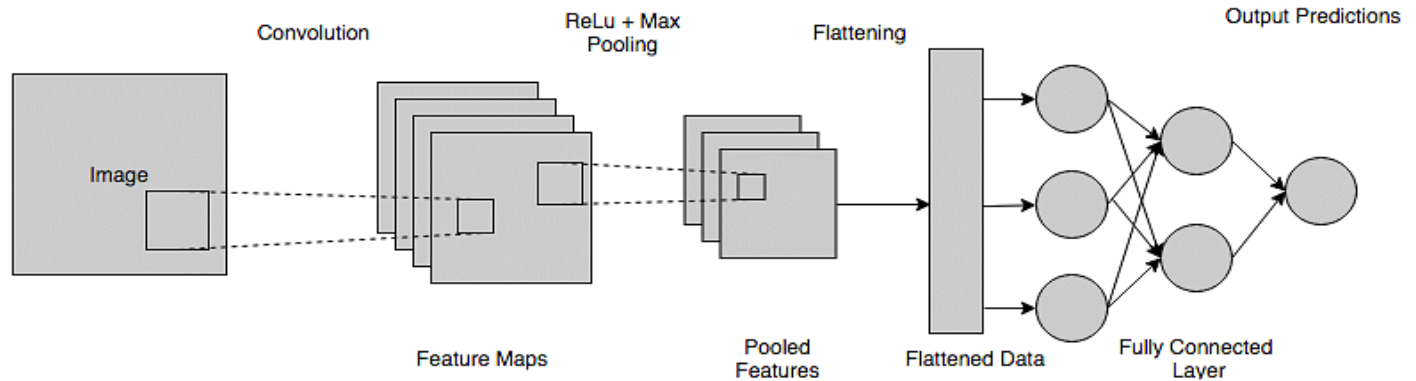
- Over a series of epochs, the model is able to distinguish between dominating and certain low-level features in images and classify them using the Softmax Classification technique.
- The information is passed through the network, and the error of prediction is calculated. The error is then backpropagated through the system to improve the prediction.

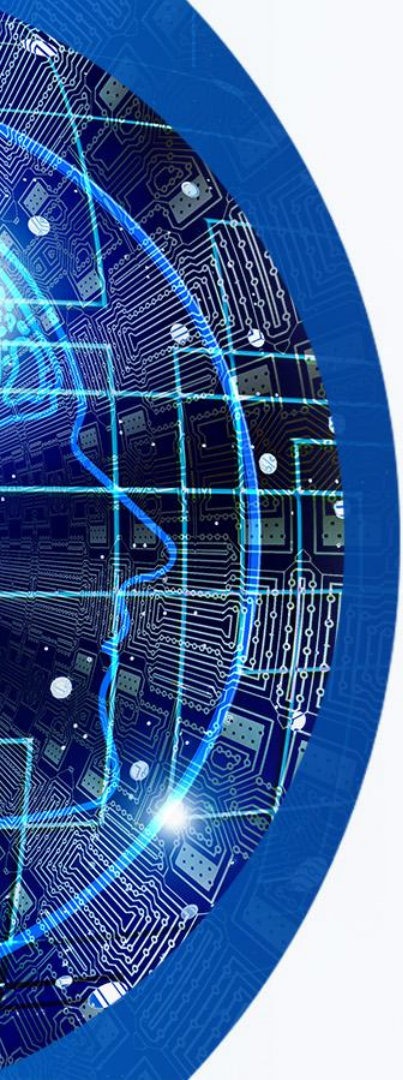
Putting it all together (A simple CNN structure)



CNN Operations

- There are certain steps/operations that are involved CNN. These can be categorized as follows:
 - Convolution $\checkmark$
 - Activation map (RELU) $\checkmark$
 - Pooling layer $\checkmark$
 - Flattening $\checkmark$
 - Fully connected layer $\checkmark$



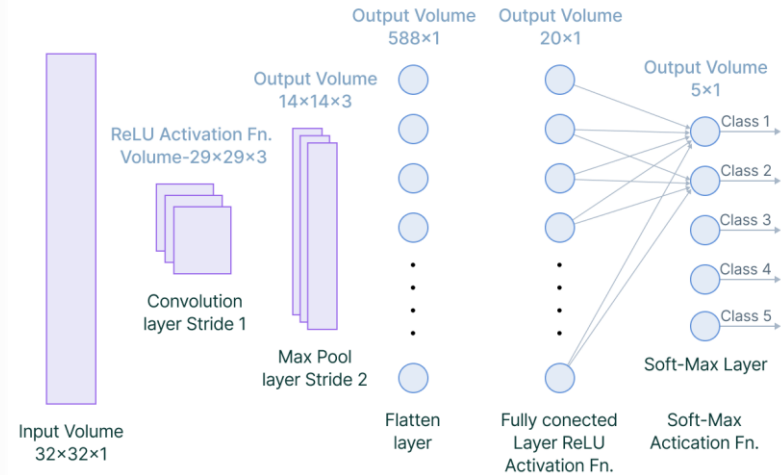


CNN Hyperparameters

- Convolution
 - Filter Size
 - Number of Filters
 - Padding
 - Stride
- Pooling
 - Window Size
 - Stride
- Fully Connected
 - Number of neurons

CNN Hyperparameters (Exmple)

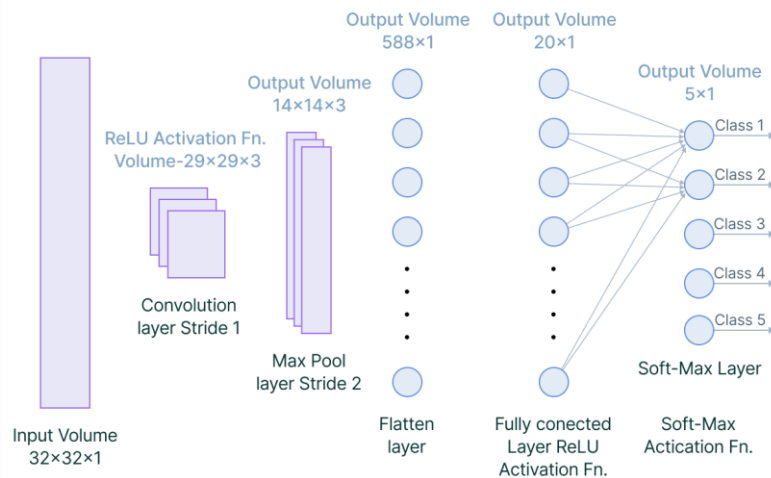
- **Input Volume:** $32 \times 32 \times 1$ (32×32 pixels, 1 color channel)
- **Convolution Layer:**
 - Stride: 1
 - Output Volume: $29 \times 29 \times 3$ (29×29 feature maps, 3 filters)
- **Max Pooling Layer:**
 - Stride: 2
 - Output Volume: $14 \times 14 \times 3$ (14×14 feature maps, 3 filters)
- **Flatten Layer:** Output Volume: 588×1 (588 features)
- **Fully Connected Layer:**
 - Output Volume: 20×1 (20 neurons)
 - Softmax Layer: Output Volume: 5×1 (5 classes)

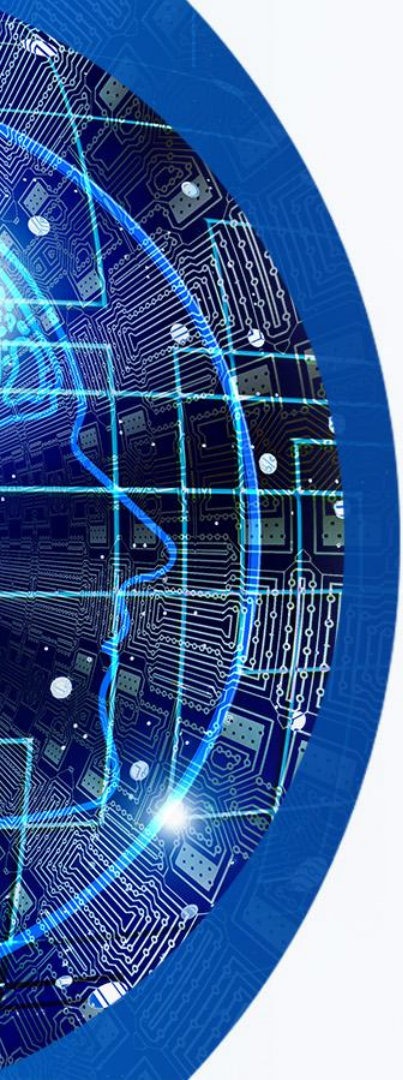


CNN Hyperparameters (Exmple)

Number of Filters and Depth:

- **Convolution Layer:** 3 filters (each filter produces one feature map)
- **Max Pooling Layer:** 3 feature maps (depth remains the same after pooling)
- **Flatten Layer:** 588 features (all features from the previous layer are flattened)
- **Fully Connected Layer:** 20 neurons (each neuron receives input from all 588 features)
- **Softmax Layer:** 5 neurons (one for each class)

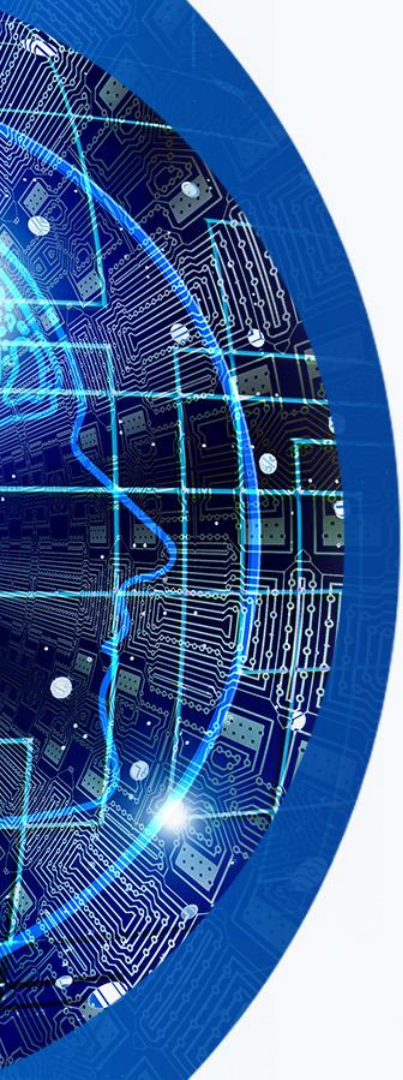




CNN Hyperparameters (Exmple)

Weights Needed to Compute:

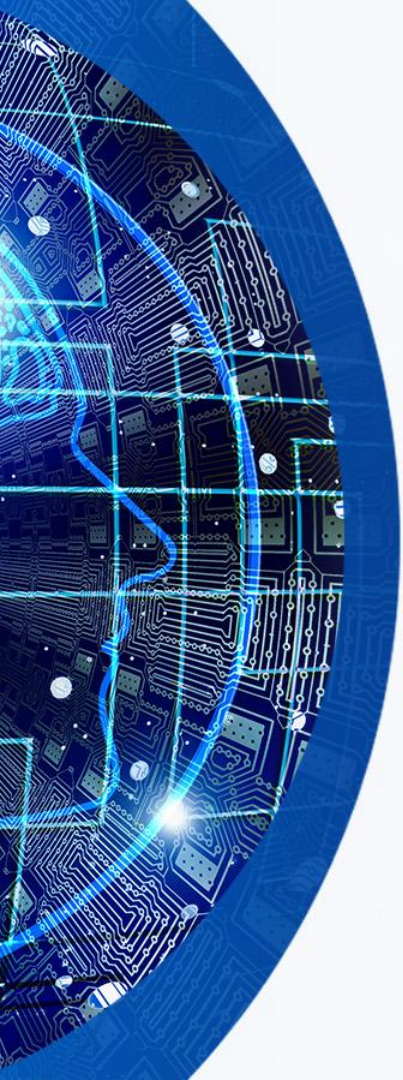
- **Convolution Layer:**
 - Each filter has a weight matrix of size (filter_size x filter_size x input_channels).
 - Assuming a 3x3 filter, each filter has $3 \times 3 \times 1 = 9$ weights.
 - With 3 filters, the total number of weights is $9 \times 3 = 27$.
- **Max Pooling Layer:**
 - No weights are required for max pooling.
- **Fully Connected Layer:**
 - Each neuron receives input from all 588 features.
 - Each connection has a weight, so the total number of weights is $588 \times 20 = 11760$.



CNN Hyperparameters (Exmple)

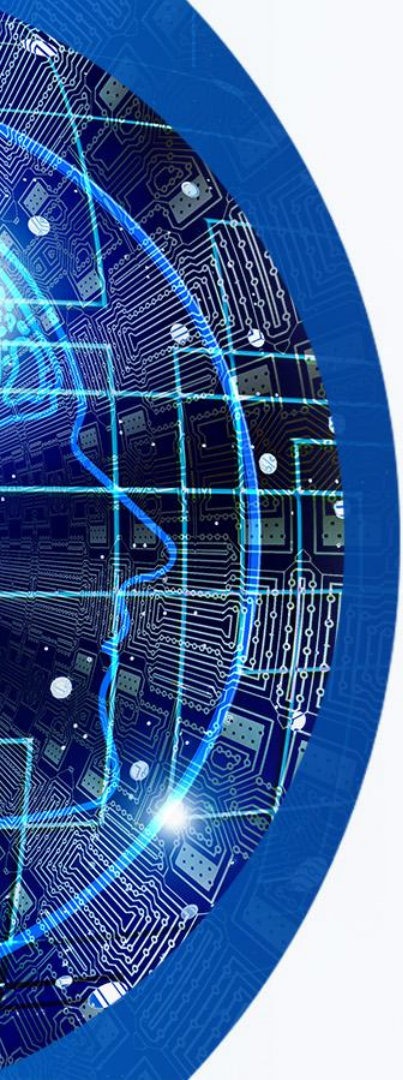
Weights Needed to Compute:

- **Softmax Layer:**
 - Each neuron receives input from all 20 neurons in the previous layer.
 - The total number of weights is $20 \times 5 = 100$.
- **Total Weights:**
 - **Adding up the weights from each layer:**
 $27 \text{ (Convolution)} + 0 \text{ (Max Pooling)} + 11760 \text{ (Fully Connected)} + 100 \text{ (Softmax)} = 11887 \text{ weights.}$



Summary of CNN entire process

- Let's summarize to see the entire process about how CNN recognizes:
 - The pixels from the image are fed to the convolutional layer that performs the **convolution operation**.
 - It results in a convolved **feature map**.
 - The convolved feature map is applied to a **ReLU function** to generate a rectified feature map.
 - The image is processed with **multiple convolutions and ReLU** layers for locating the features.
 - Different **pooling layers** with various filters are used to identify specific parts of the image.
 - The pooled feature map is **flattened** and fed to a **fully connected layer** to get the final output.

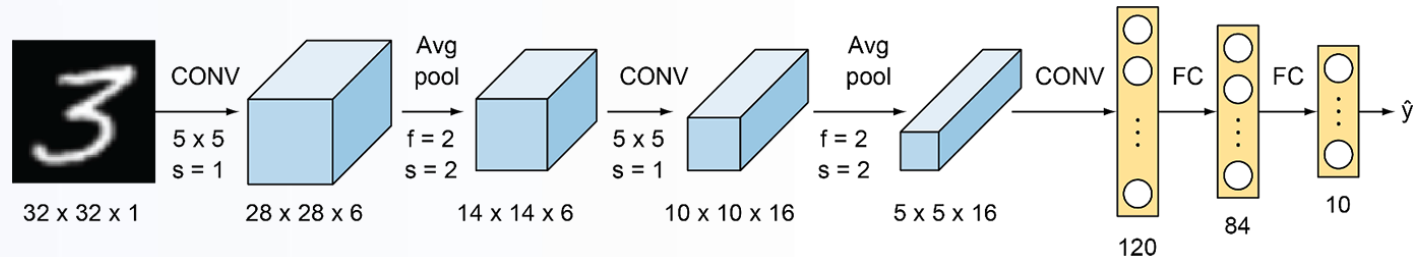


Popular CNN Networks

- We will look at the following popular networks:
 1. LeNet-5
 2. AlexNet
 3. VGG-16
 4. ResNet
 5. Inception

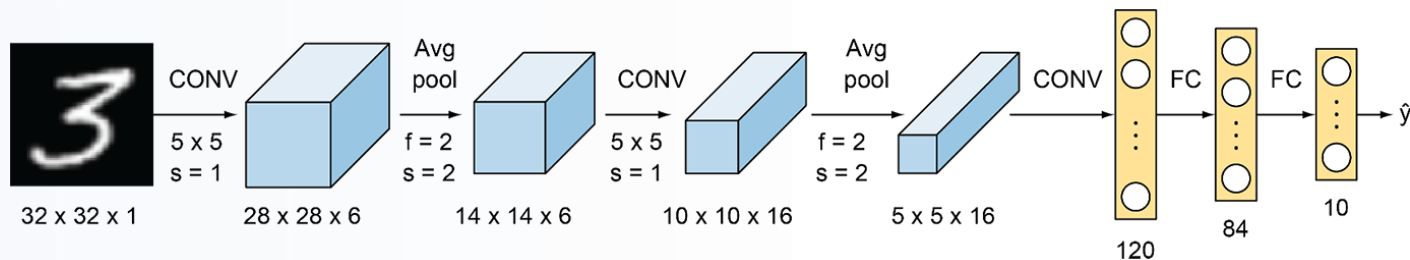
LeNet-5

- LeNet-5 is one of the earliest pre-trained models proposed by Yann LeCun in 1998. It is used for recognizing handwritten and machine-printed characters.
- It takes a grayscale image as input. Once we pass it through a combination of convolution and pooling layers, the output will be passed through fully connected layers and classified over 10 classes, representing the likelihood of the input image belonging to each class (0-9 digits).



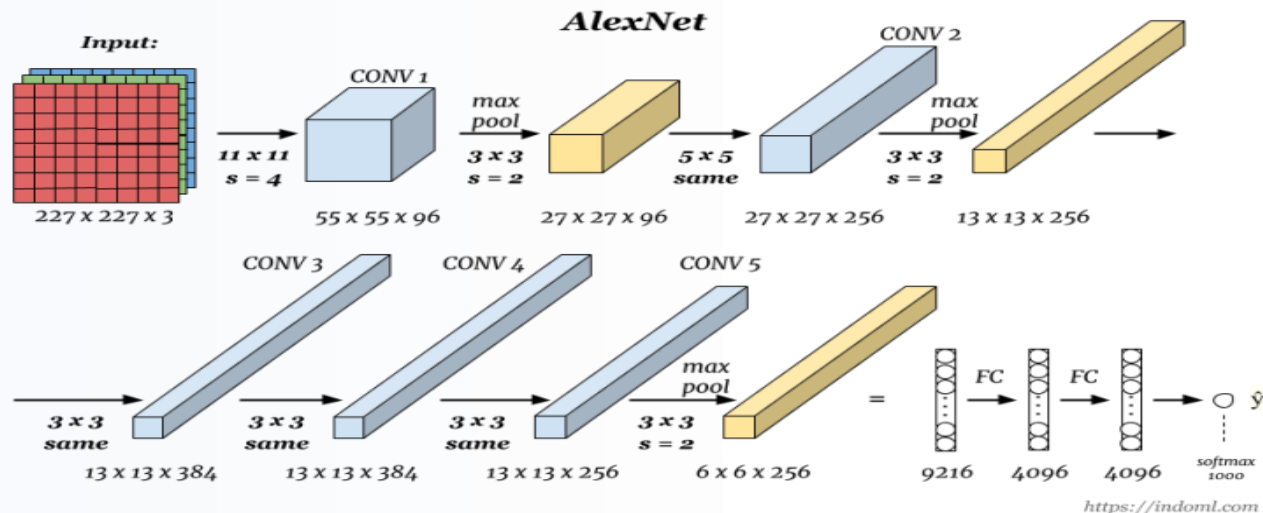
LeNet-5

- **Handwritten Digit Recognition:** It achieved high performance on the MNIST handwritten digit recognition dataset.
- **The total number of parameters in LeNet-5 are:**
 - **Parameters:** 60k
 - **Layers flow:** Conv -> Pool -> Conv -> Pool -> FC -> FC -> Output
 - **Activation functions:** Sigmoid/tanh and ReLu



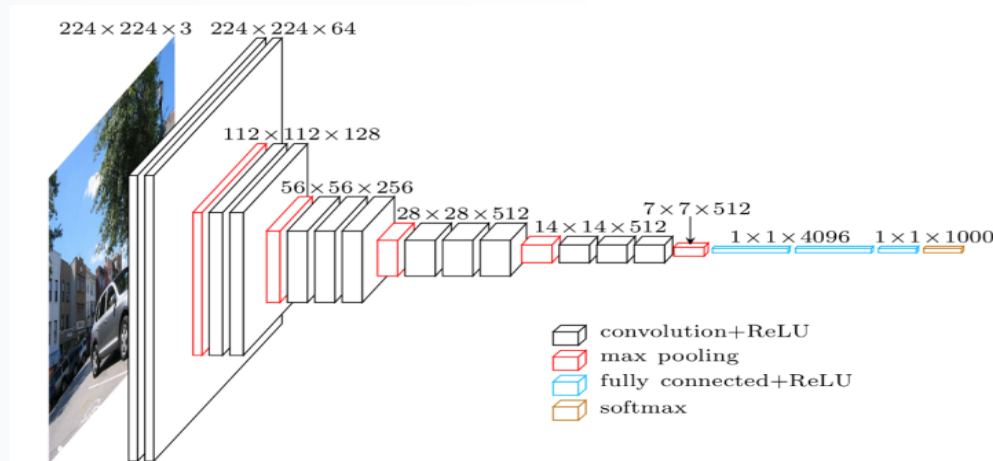
AlexNet

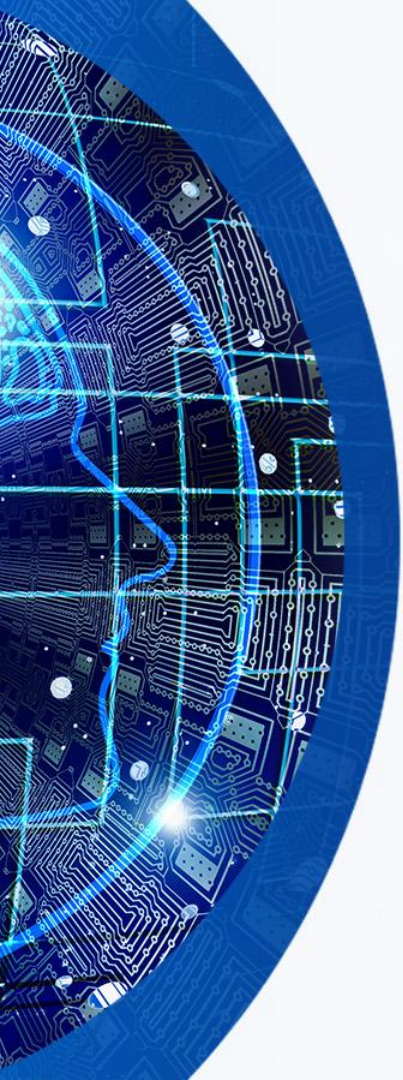
- It was developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton in 2012
- This network is similar to LeNet-5 with just more convolution and pooling layers and takes coloured images:
- **Parameters:** 60 million
- **Activation function:** ReLu



VGG-16

- The "16" in VGG-16 refers to the number of layers with weights in the network.
- The output layer uses a softmax function to produce a probability distribution over 1000 different object categories.
- As it is a bigger network, the number of parameters are also more.
- Parameters: 138 million





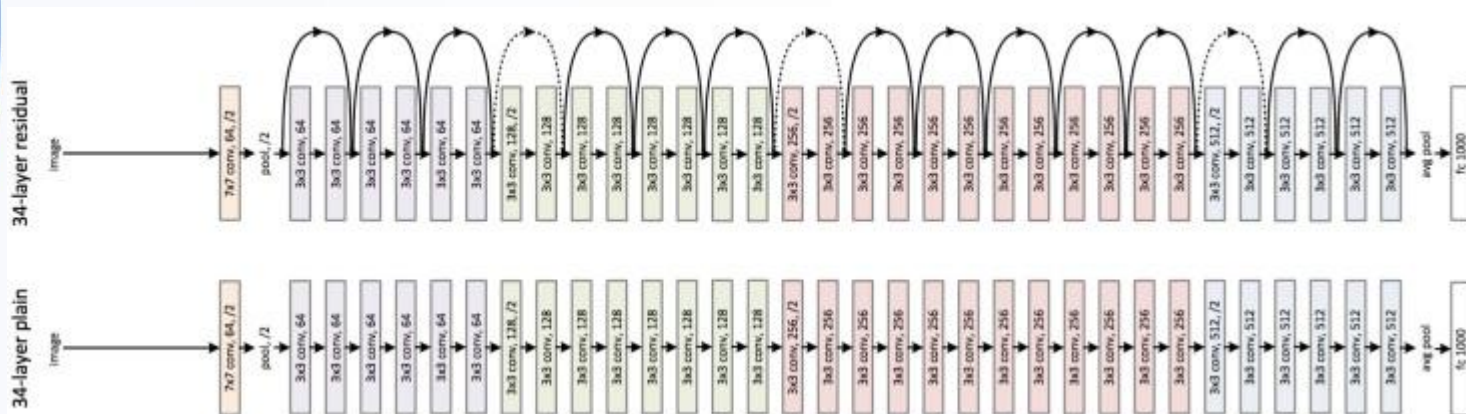
ResNet

- Training very deep networks can lead to problems like vanishing and exploding gradients
 - When the number of activation functions increases in the network, the gradient of the loss function becomes extremely small (almost zero), making it hard to train the model. It also results in loss of information.
- This brought the building of ResNet. The idea of residual connections, **which created shortcuts within CNN layers, allowing the model to skip middle layers during backpropagation.**

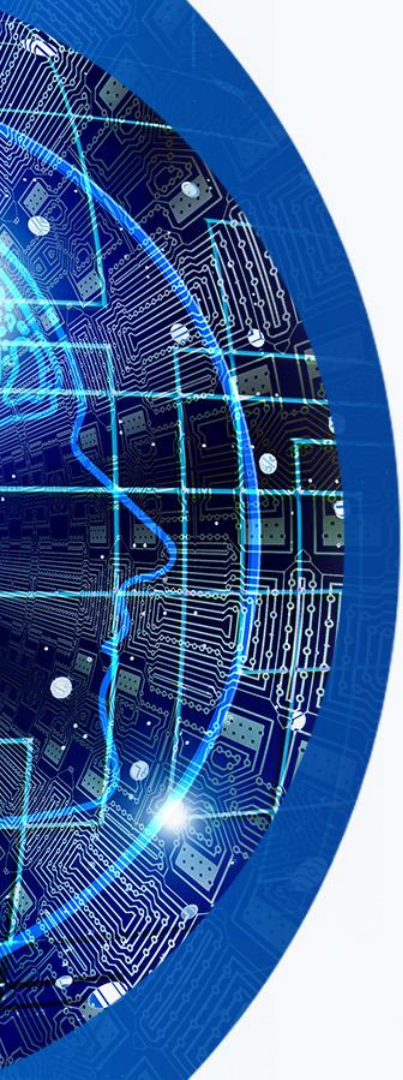
Why ResNets Work?

- **Residual Blocks:** These blocks contain multiple convolutional layers, followed by a shortcut connection that adds the input to the output of the block.

ResNet



Regular or Plain Network

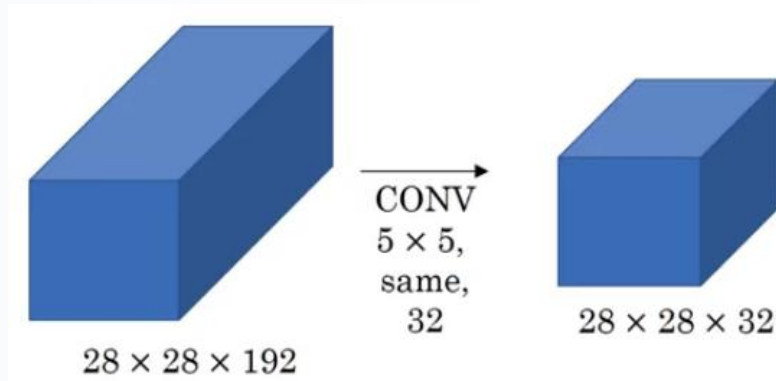


Benefits of ResNet

- ResNets can be significantly deeper than traditional CNNs without suffering from vanishing gradients.
- ResNets have achieved high performance on various image recognition tasks.
- The residual connections help in faster training and convergence.

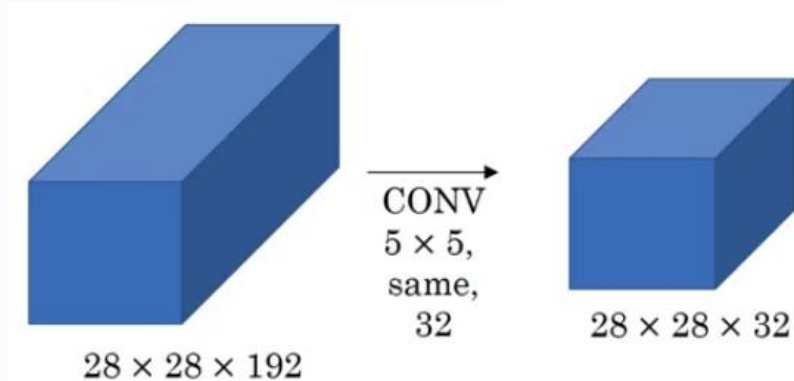
The Motivation Behind Inception Networks

- Why are we using all these filters instead of using just a single filter size, say 5×5 ? Let's look at how many computations would arise if we would have used only a 5×5 filter with padding on our input:



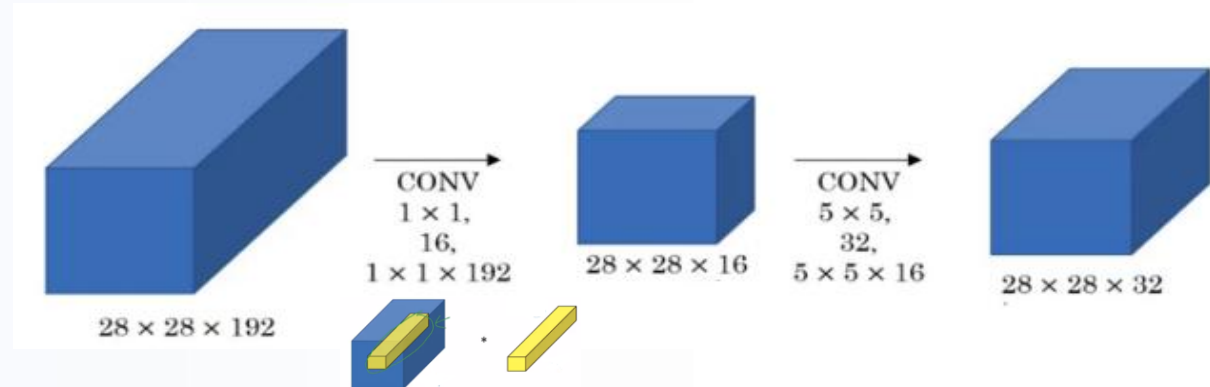
The Motivation Behind Inception Networks

- Number of multiplies = $28 * 28 * 32 * 5 * 5 * 192 = 120$ million!
Can you imagine how expensive performing all of these will be?
- Using 1×1 convolution and then a 5×5 convolution will reduce the computation to about 1/10 of that



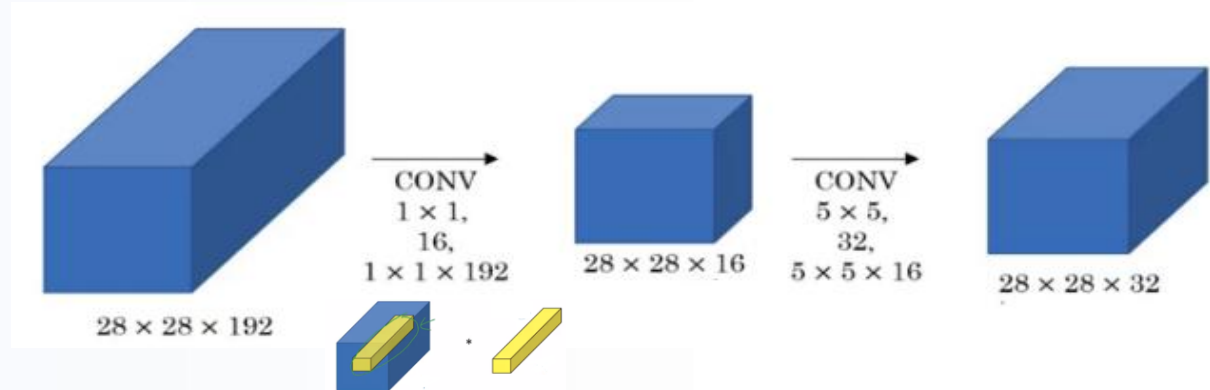
The Motivation Behind Inception Networks

- Number of multiplies for first convolution = $28 * 28 * 16 * 1 * 1 * 192 = 2.4$ million
- Number of multiplies for second convolution = $28 * 28 * 32 * 5 * 5 * 16 = 10$ million
- Total number of multiplies = 12.4 million
- **A significant reduction. This is the key idea behind inception.**



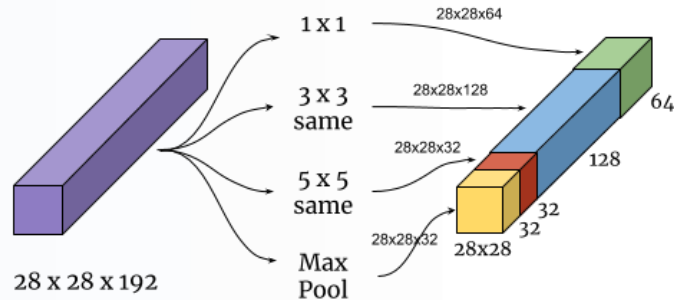
The Motivation Behind Inception Networks

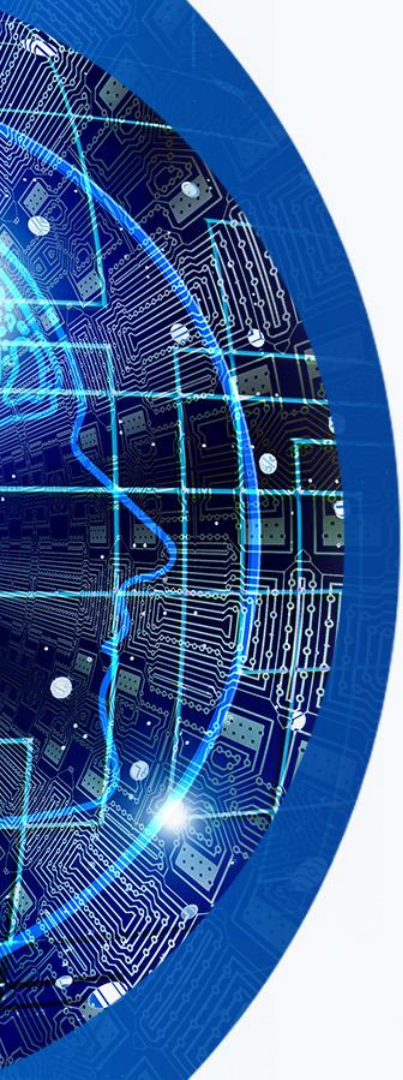
- The core idea behind Inception Networks is to combine different sized filters to extract features at multiple scales.
- Each Inception module consists of multiple convolutional layers with different filter sizes (e.g., 1×1 , 3×3 , and 5×5) and a pooling layer. The outputs of these layers are concatenated to form the output of the module.



The Motivation Behind Inception Networks

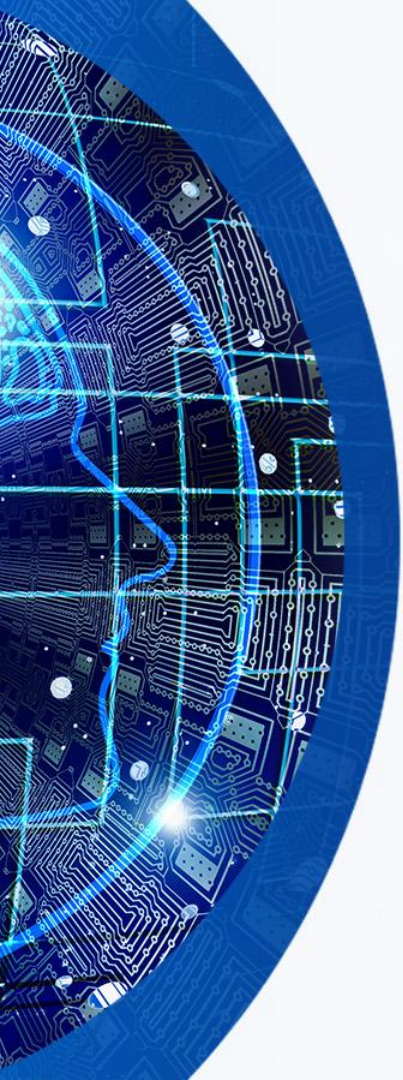
- Suppose we have a $28 \times 28 \times 192$ input volume. **Instead of choosing what filter size to use**, or whether to use convolution layer or pooling layer, **inception uses all of them and stacks all the outputs**.
 - These filters work together, examining the image from different perspectives.
 - Each filter extracts specific features, like edges, textures, or shapes.
 - The information from all the filters is combined to form a more complete understanding of the image.





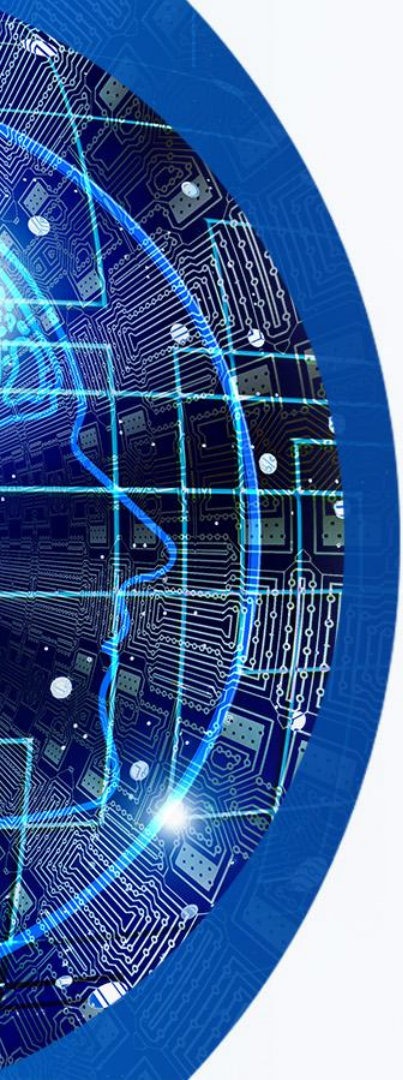
Transfer Learning

- **Transfer Learning** is a machine learning technique where a pre-trained model, typically trained on a large dataset like ImageNet, is reused as a starting point for a new task.
- A pre-trained model is trained on a large dataset to learn general features, such as edges, textures, and shapes.
- This technique is particularly useful in computer vision, where it can accelerate training and improve performance, especially when dealing with limited data.



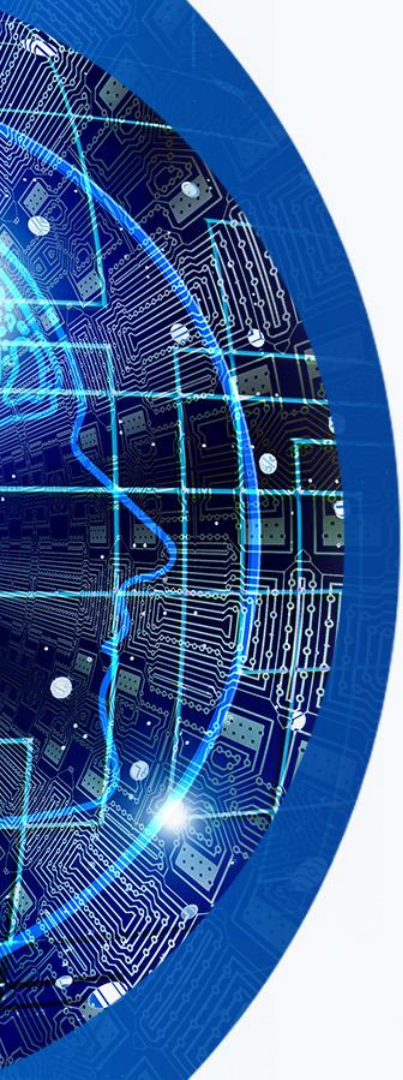
Types of Transfer Learning:

- **Feature Extraction:** Only the pre-trained model's features are used, and a new classifier is trained on top of these features.
- **Fine-tuning:** The pre-trained model's weights are fine-tuned on the new dataset, allowing the model to adapt to the specific characteristics of the new task.



Transfer Learning Advantages

- **Feature Extraction:** Pre-trained models, like those used in ImageNet, have learned to extract powerful and generalizable features from images. **By transferring these learned features to a new task**, we can significantly improve the performance of our model, especially when dealing with smaller datasets.
- **Faster Training:** Instead of training a model from scratch, we can start with a pre-trained model and fine-tune it on our specific task. This can significantly reduce training time and computational resources.
- **Better Performance:** By knowing features learned from a large dataset, transfer learning can lead to more accurate and robust models.



Applications of Transfer Learning:

- **Computer Vision:** Object detection, image classification, image segmentation
- **Natural Language Processing:** Text classification, sentiment analysis, machine translation
- **Speech Recognition:** Speech-to-text, speaker identification

Object Recognition

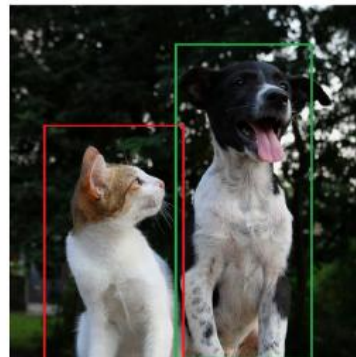
- **Object Recognition** as an umbrella term for three tasks (Image Classification, Object Localization, and Object Detection).



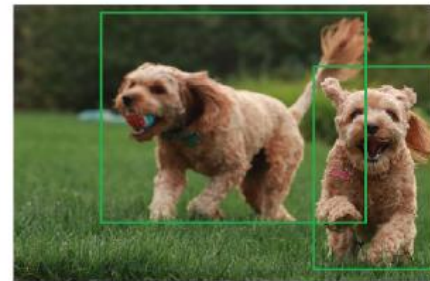
Image belongs
to Dog class



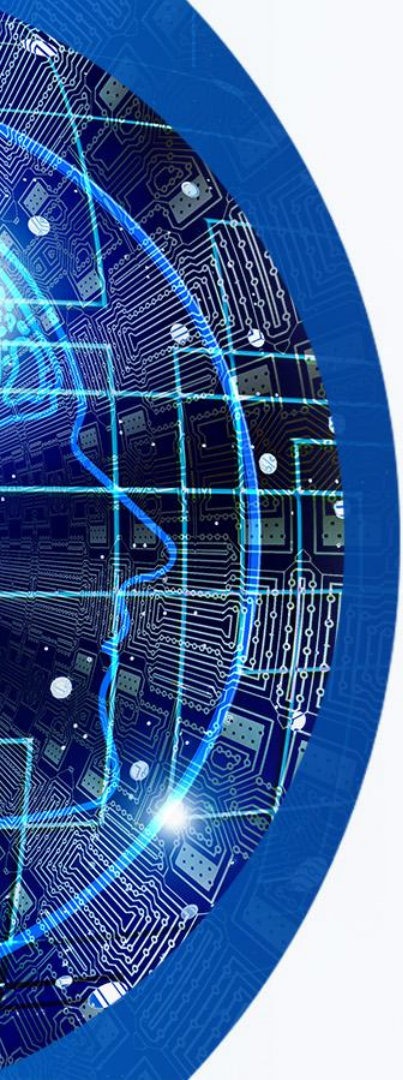
Image belongs
to Cat class



cat detected
on the image dog detected
on the image

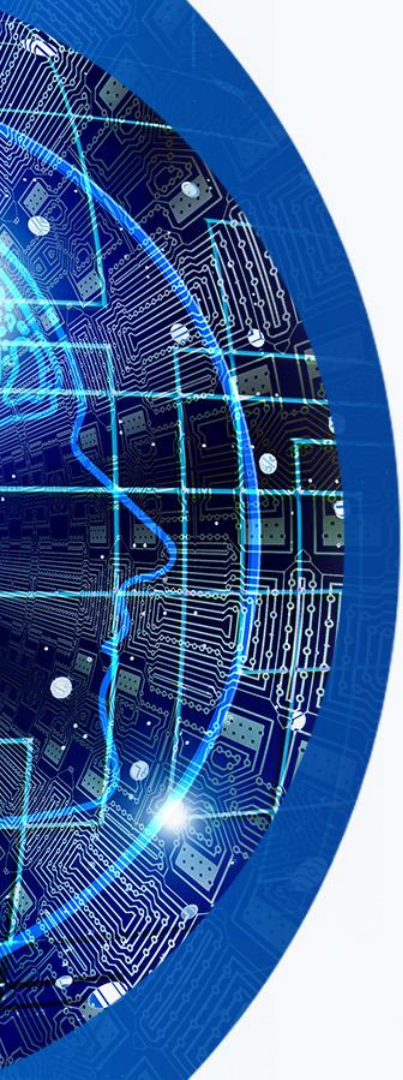


dog detected
on the image dog detected
on the image



Object Recognition

- **Image Classification:** Predict the type or class of an object in an image.
 - Input: An image with a single object, such as a photograph.
 - Output: A class label (e.g. one or more integers that are mapped to class labels).
- **Object Localization:** Locate the presence of objects in an image and indicate their location with a bounding box.
 - Input: An image with one or more objects, such as a photograph.
 - Output: One or more bounding boxes (e.g. defined by a point, width, and height).

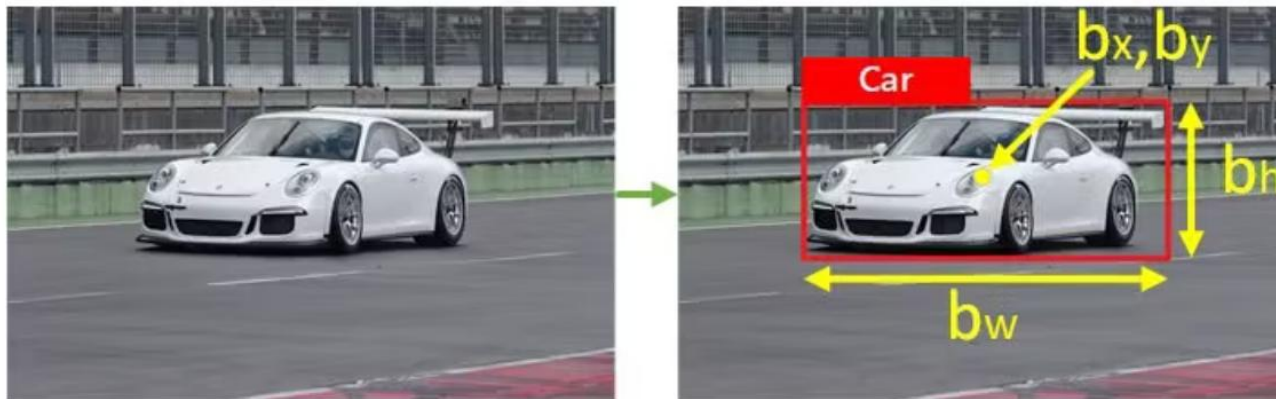


Object Recognition

- **Object Detection:** Locate the presence of objects with a bounding box and types or classes of the located objects in an image.
 - Input: An image with one or more objects, such as a photograph.
 - Output: One or more bounding boxes (e.g. defined by a point, width, and height), and a class label for each bounding box.

Object Localization

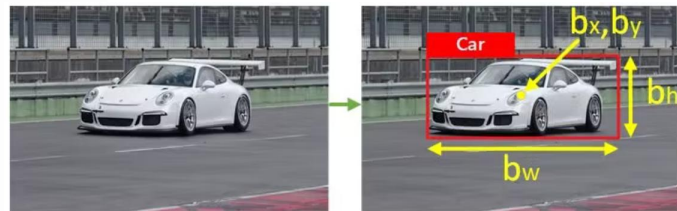
- An object localization model is similar to a classification model. But the trained localization model also predicts where the object is located in the image by drawing a bounding box around it.
 - For example, a car is located in the image below. The information of the bounding box, center point coordinate, width and, height is also included in the model output.



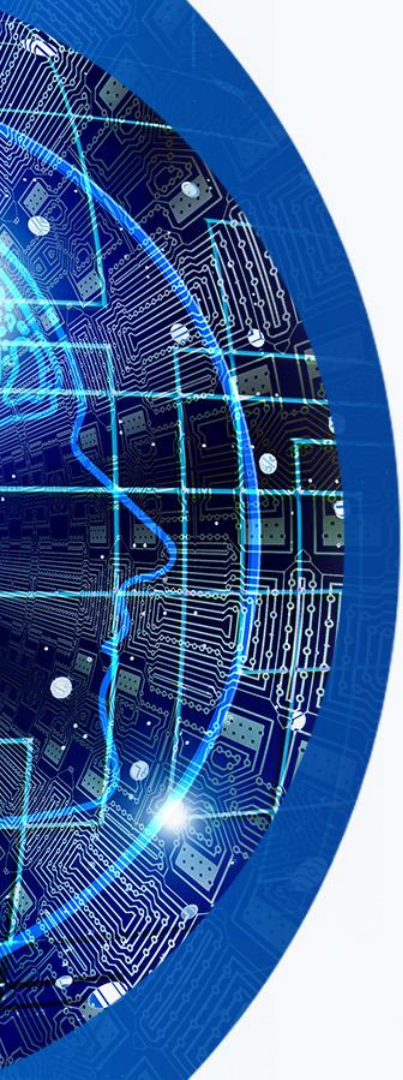
Object Detection

- Assume we have 3 types of targets to detect

1. Train
2. car
3. motorcycle



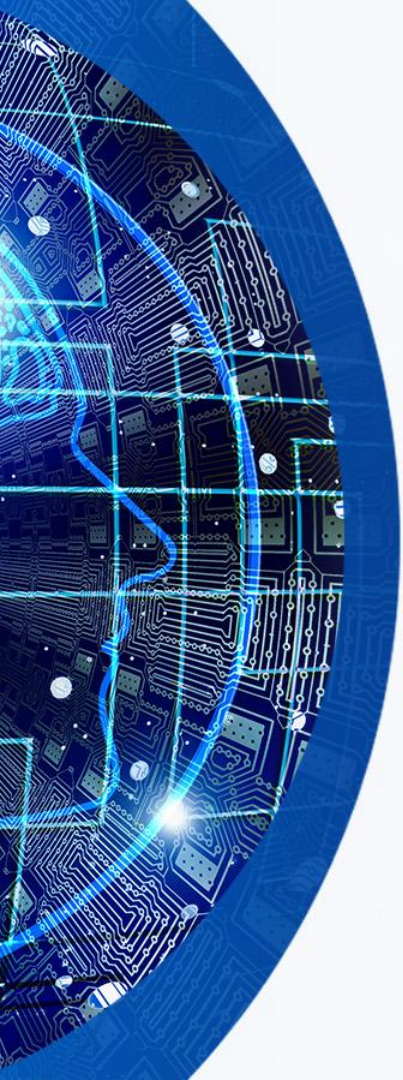
- For the classification model, the output will be a list of 3 numbers representing the probability for each class. For example, the previous image with only a car inside the output may look like $[0.001, 0.998, 0.001]$. The second class which is the car has the largest probability.



Object Detection

- The output of the localization model will include the information of the bounding box, so the output will look like

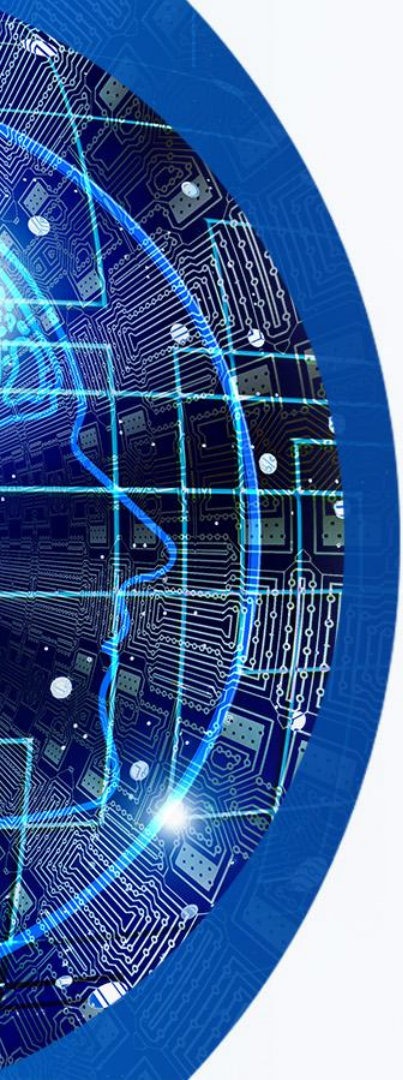
$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$



Object Detection

- p_c : being 1 means any object is detected. If it is 0, the rest of output will be ignored.
- b_x : x coordinate, the center of the object corresponding to the upper left corner of the image
- b_y : y coordinate, the center of the object corresponding to the upper left corner of the image
- b_h : height of the bounding box
- b_w : width of the bounding box
- c_1, c_2, c_3 , in case there is an object, tell if the object is part of class 1, 2 or 3

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

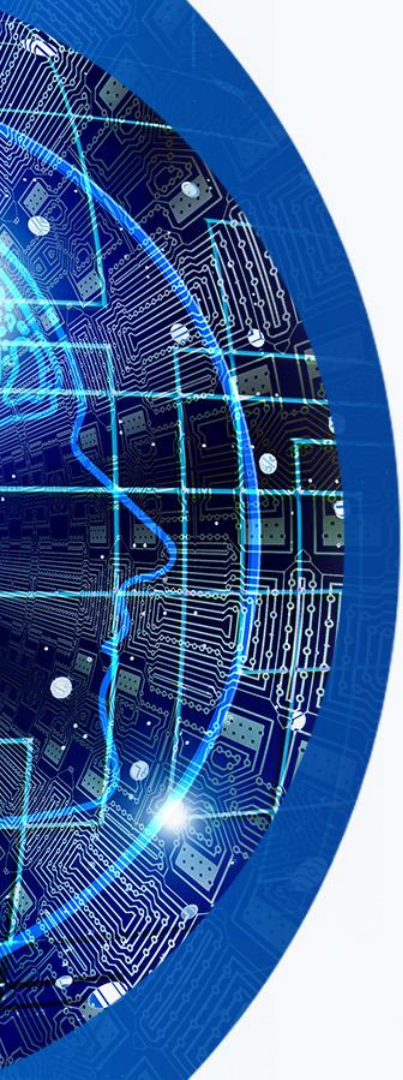


Object Detection

- In case the network doesn't detect an object, the output is simply:

$$\begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix}$$

- Where the question marks are placed in the rest of the positions that don't provide any meaning in this case

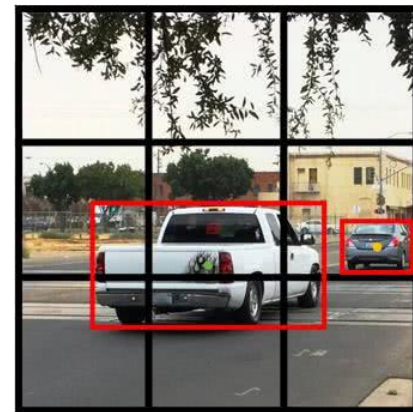


Object Detection

- **Object localization is very slow** since we have to predict on lots of regions and try lots of box sizes to have a more accurate result.
- It is **computationally intensive**, so it is hard to achieve good real-time object localization performance as required in an application like self-driving cars.
- The position of the bounding box **is not going to be very accurate** based on how we choose the stride and how many different sizes of bounding boxes to try.

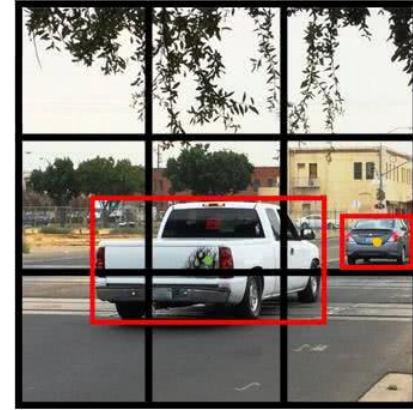
Object Detection

- **You Only Look Once (YOLO):** directly predicts bounding box coordinates and class probabilities in a single step.
 - In contrast to traditional two-stage object detection methods, which involve separate region proposal and object classification stages.
 - YOLO eliminates this two-step process, making it significantly faster.



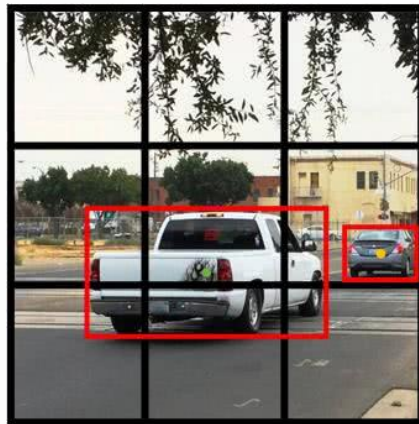
Object Detection

- YOLO divides the input image into a grid of cells. Each cell predicts a fixed number of bounding boxes and their corresponding class probabilities.
- After predicting multiple bounding boxes for each object, YOLO uses a technique to eliminate redundant detections and select the most confident ones.



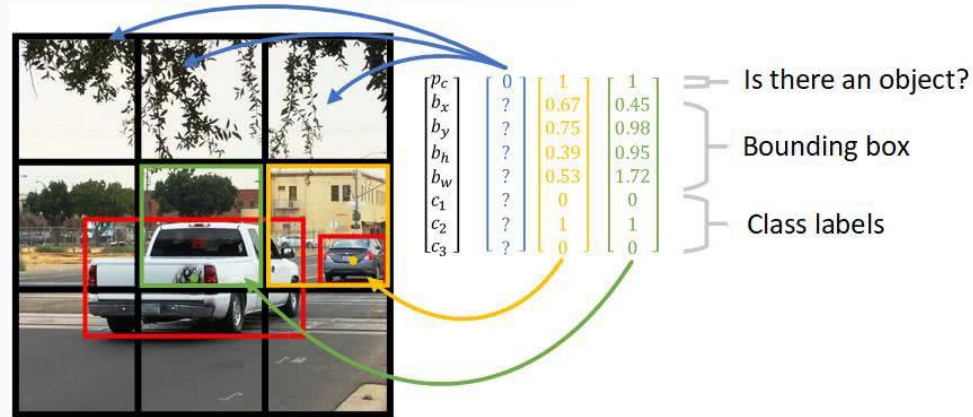
Object Detection

- For the rightmost car, it's easy. It belongs to the middle right cell since its bounding box is inside that grid cell.
- For the truck in the middle of the image, its bounding box intersects with several grid cells. The YOLO algorithm takes the middle point of the bounding box and associates it to the grid cell containing it.

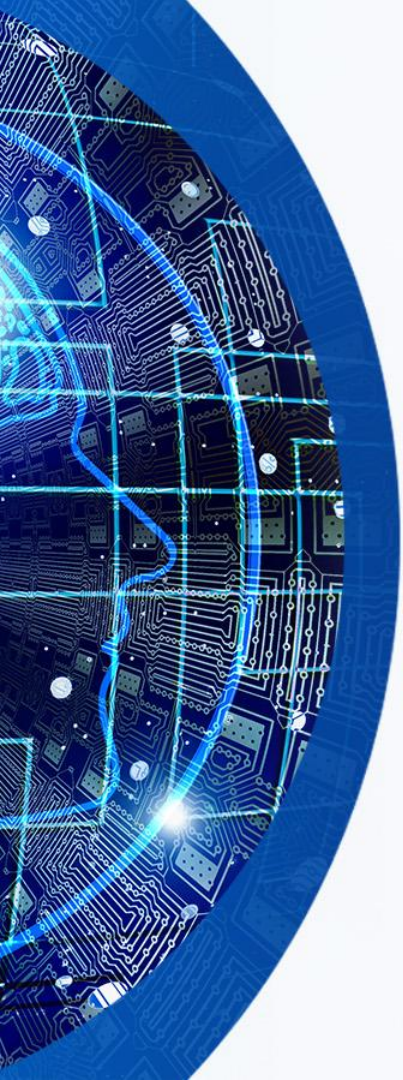


Object Detection

- As a result, here are the output labels for each grid cell.



- YOLO is faster than other object detection algorithms, making it suitable for real-time applications.
- Its single-stage approach simplifies the detection process.



Thank You